

Relaciones entre tablas y mapeo interno

Recorrido **UML** → **TypeORM** → **ER** y, después, cómo cerrar la capa HTTP con el **ValidationPipe** de Nest y **class-validator**: instalación de paquetes, activación global, DTO y decoradores típicos (sin duplicar reglas ya impuestas en la base por TypeORM/constraints donde aplique).

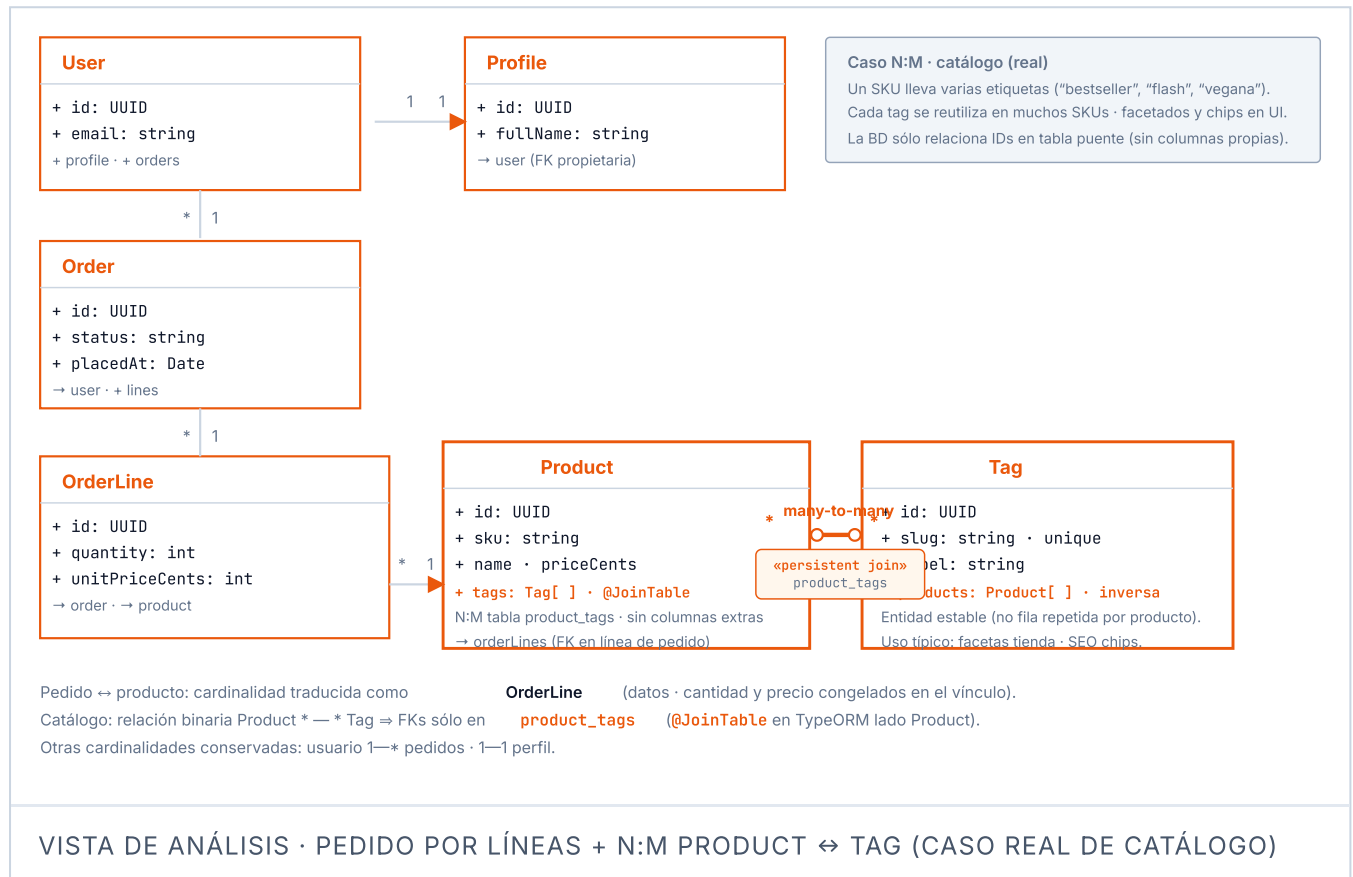
Ideas clave

- **Diagrama de clases:** define multiplicidades y roles antes de escribir decorators.
- **TypeORM:** cada **@Entity** y relación se materializa en FKs o tablas según el lado propietario.
- **Más abajo:** validación HTTP con Nest (**class-validator** + **ValidationPipe**).

Clic en **Bajar** abajo (o imprimir / guardar como PDF). Todo el contenido sigue en esta misma página.

Diagrama de clases (UML)

Modelo orientado a objetos de un flujo real: un **User** tiene opcionalmente un **Profile**; un usuario realiza muchos **Order**; cada pedido contiene muchas **OrderLine**; cada línea apunta a un **Product** con cantidad y precio histórico. En paralelo en el mismo dominio típico, **muchos productos comparten etiquetas** (*bestseller*, *envío gratis*): **Product** *—* **Tag**, muchos a muchos clásicos con tabla puente **product_tags** (sin datos de negocio en el vínculo más allá del par IDs).



Mismo modelo en entidades

Cada asociación del diagrama se traduce en `@OneToOne`, `@OneToMany` / `@ManyToOne` u `@ManyToMany`. El vínculo pedido–producto pasa por **OrderLine** (cantidad · precio congelados). El vínculo **producto** ↔ **tag** es N:M explícito con `@JoinTable` sobre `product_tags`.

CÓDIGO (RESUMIDO; IMPORTS OMITIDOS)

```
@Entity('users')
export class User {
  @PrimaryGeneratedColumn('uuid')
  id: string;

  @Column({ unique: true })
  email: string;

  @OneToMany(() => Order, (o) => o.user)
  orders: Order[];

  @OneToOne(() => Profile, (p) => p.user)
  profile: Profile;
}

@Entity('profiles')
export class Profile {
  @PrimaryGeneratedColumn('uuid')
  id: string;

  @Column({ length: 200 })
  fullName: string;

  @OneToOne(() => User, (u) => u.profile)
  @JoinColumn({ name: 'user_id' })
  user: User;
}

@Entity('orders')
export class Order {
  @PrimaryGeneratedColumn('uuid')
  id: string;

  @Column({ length: 32 })
  status: string;

  @CreateDateColumn({ name: 'placed_at' })
  placedAt: Date;
```

```

    @ManyToOne(() => User, (u) => u.orders, { nullable: false })
    @JoinColumn({ name: 'user_id' })
    user: User;

    @OneToMany(() => OrderLine, (line) => line.order)
    lines: OrderLine[];
}

@Entity('products')
export class Product {
    @PrimaryGeneratedColumn('uuid')
    id: string;

    @Column({ unique: true })
    sku: string;

    @Column()
    name: string;

    @Column({ type: 'int' })
    priceCents: number;

    @OneToMany(() => OrderLine, (line) => line.product)
    orderLines: OrderLine[];

    @ManyToMany(() => Tag, (tag) => tag.products)
    @JoinTable({
        name: 'product_tags',
        joinColumn: { name: 'product_id', referencedColumnName: 'id' },
        inverseJoinColumn: { name: 'tag_id', referencedColumnName: 'id' },
    })
    tags: Tag[];
}

@Entity('tags')
export class Tag {
    @PrimaryGeneratedColumn('uuid')
    id: string;

    @Column({ unique: true })
    slug: string;

    @Column({ length: 96 })
    label: string;

    @ManyToMany(() => Product, (p) => p.tags)
    products: Product[];
}

@Entity('order_lines')
export class OrderLine {

```

```

@PrimaryGeneratedColumn('uuid')
id: string;

@Column({ type: 'int' })
quantity: number;

@Column({ type: 'int', name: 'unit_price_cents' })
unitPriceCents: number;

@ManyToOne(() => Order, (o) => o.lines, { onDelete: 'CASCADE' })
@JoinColumn({ name: 'order_id' })
order: Order;

@ManyToOne(() => Product, (p) => p.orderLines)
@JoinColumn({ name: 'product_id' })
product: Product;
}

```

Lados propietarios de la FK: `Profile.user`, `Order.user`, `OrderLine.order`, `OrderLine.product`. El resto son inversos y no añaden columnas en sus tablas. La tabla puente `product_tags` la “posee” el lado marcado con `@JoinTable` (`Product.tags`).

Product ↔ Tag con @JoinTable

Caso real: etiquetas reutilizables (*vegan*, *bestseller*, *envío gratis*) aplicadas a muchos productos. La relación no necesita columnas de negocio propias (solo qué producto tiene qué tag). Ahí **@ManyToMany** + tabla **product_tags** es apropiado. Si mañana necesitas **assignedAt** o prioridad por producto, migrás a una entidad **ProductTag** explícita como hicimos con **OrderLine**.

REGLA PRÁCTICA

- **Sin datos en el vínculo** → **@ManyToMany** + **@JoinTable** (o tabla intermedia sin entidad, según convención del equipo).
- **Con datos en el vínculo** (cantidad, precio histórico, orden de visualización) → entidad intermedia con dos **@ManyToOne**.



TypeORM crea la tabla puente product_tags con las dos FK.
Lado propietario del join: Product.tags (@JoinTable).

ER · ASOCIACIÓN N:M CLÁSICA (CATÁLOGO)

PERSISTENCIA TÍPICA

```

// import { In } from 'typeorm';

// Cargar producto con etiquetas
const p = await productRepo.findOne({
  where: { sku: 'SKU-001' },
  relations: { tags: true },
});

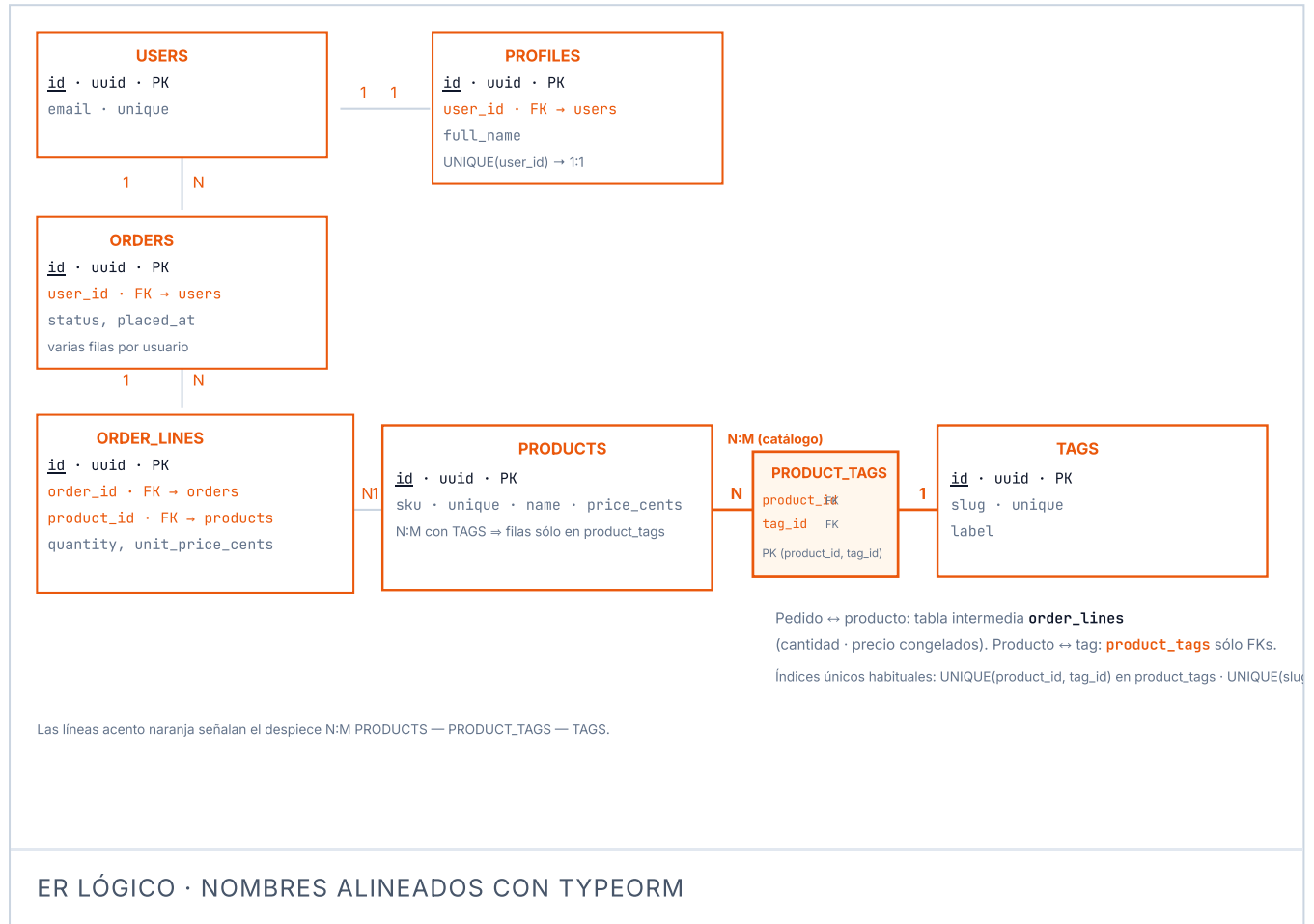
// Asignar tags existentes (save actualiza la tabla puente)

```

```
p.tags = await tagRepo.findBy({ slug: In(['bestseller', 'envio-gratis']) });  
await productRepo.save(p);
```

Diagrama entidad-relación

Rectángulos = entidades (tablas); subrayado = clave primaria; etiquetas **FK** = foráneas. Misma semántica que en UML: pedido y catálogo vía **order_lines**; además, **productos** y **tags** enlazan por **product_tags** (N:M pura del bloque anterior).



MISMO ESQUEMA EN GRILLAS

USERS id email	PROFILES id user_id → users full_name	ORDERS id user_id → users status, placed_at
ORDER_LINES id order_id → orders product_id → products quantity, unit_price_cents	PRODUCTS id sku, name, price_cents	TAGS id slug · unique, label

PRODUCT_TAGS

product_id → products

tag_id → tags

(poseída por
Product.tags)

VALIDACIÓN · NESTJS

Dónde entra **class-validator**

En Nest, los cuerpos y query strings llegan como JSON plano. Los **DTO** (clases plain) pueden decorarse con restricciones; el **ValidationPipe** usa **class-validator** para comprobarlas antes de ejecutar tu controlador. Opcionalmente, **class-transformer** convierte ese JSON en instancias de clase y permite **@Type(() => ...)** en objetos anidados y arrays.

Flujo habitual

- **Petición HTTP** → interceptor / pipe ejecuta antes del handler.
- **ValidationPipe** instancia/transforma el DTO y ejecuta los decoradores (**@IsEmail()**, etc.).
- Si algo falla, Nest devuelve **400 Bad Request** con lista de errores (modificable con un filtro).

Paquetes npm

Instalá ambos si vas a usar `ValidationPipe` con `{ transform: true }`, validación anidada o transformación plain → clase.

COMANDOS

```
npm i class-validator class-transformer
```

Versión Yarn / pnpm: `yarn add class-validator class-transformer`

TipoScript: dejá `experimentalDecorators` y `emitDecoratorMetadata` activos en tu `tsconfig.json` como indica la guía oficial de Nest; sin eso, los pipes basados en metadatos no funcionan igual.

ValidationPipe en main.ts

Patrón recomendado: registrar el pipe **una sola vez** en **bootstrap** con `app.useGlobalPipes(...)`. Así aplica a todos los controladores (salvo que uses otro pipe a nivel ruta o método).

Dónde está en este documento: archivo `presentacion-typeorm-relaciones.html`, sección con badge **Activación global** (`#sec-cv-pipe`) — el fragmento de `main.ts` va justo arriba del acordeón “Qué hace cada propiedad”.

```
import { ValidationPipe } from '@nestjs/common';
import { NestFactory } from '@nestjs/core';
import { AppModule } from './app.module';

async function bootstrap() {
  const app = await NestFactory.create(AppModule);
  app.useGlobalPipes(
    new ValidationPipe({
      transform: true,
      whitelist: true,
      forbidNonWhitelisted: true,
      transformOptions: {
        enableImplicitConversion: true, // opcional · ve explicación abajo
      },
    }),
  );
  await app.listen(3000);
}
bootstrap();
```

QUÉ HACE CADA PROPIEDAD

- **transform: true**

Habilita **class-transformer** dentro del pipe: el body (objeto literal JSON) se convierte en una **instancia de la clase del DTO** antes de validar. Sin esto suele quedar un objeto plano aunque el tipado diga `CreateUserDto`. Con esta opción encendida funcionan mejor `@Type(() => ...)` (anidados), defaults en la clase y el flujo completo “transformar → whitelist → validar”.

- **whitelist: true**

Tras la conversión, el pipe **saca del objeto** las propiedades que **no tienen decorador de validación/exposición reconocido** (p. ej. ningún `@IsString()`,

`@IsOptional()`, `@Allow()`, etc.). Evita **asignación masiva**: un campo como `isAdmin` no entra si no está en el DTO. Cuidado: si te olvidás decorar un campo que sí querés leer, también se eliminará — agregá validadores o `@Allow()` donde corresponda.

- **`forbidNonWhitelisted: true`**

Complemento de `whitelist`: si el cliente manda **propiedades sobrantes** (las que el pipe descartaría), la petición falla con **400 Bad Request** en lugar de tirarlas sin avisar. Útil para detectar clientes desactualizados o intentos de inyectar datos. Con `false`, los extras se ignoran y, si el resto valida, la solicitud sigue.

- **`transformOptions: { enableImplicitConversion: true }`**

Opciones del transformador de Nest. Con `enableImplicitConversion`, se intentan **coerciones** usando metadatos del tipo (reflect), p. ej. string `"10"` a número en un `@IsInt()`. Muy práctico para **query params** y campos que llegan siempre como string. Revisá edge cases: si algo se vuelve ambiguo, es más explícito usar `@Type(() => Number)` en la propiedad además o en lugar de confiar solo en la conversión implícita.

Opcional en producción: `disableErrorMessage: true` en las mismas opciones del `ValidationPipe` hace que los 400 lleven menos texto de reglas (menos detalle expuesto al cliente); en local conviene `false` para depurar.

CreateUserDto alineado con el dominio de la presentación

Misma idea que la entidad **User** vista antes: sólo exponé en el POST lo necesario (**email**); ignorá PK y relaciones cargadas hasta que hagas otros endpoints.

```
import {
  IsEmail,
  IsOptional,
  IsString,
  MaxLength,
  MinLength,
} from 'class-validator';

export class CreateUserDto {
  @IsEmail()
  @MaxLength(320)
  email: string;

  @IsOptional()
  @IsString()
  @MinLength(2)
  @MaxLength(200)
  fullName?: string; // podrías mapearlo luego al Profile en servicio
}
```

USO EN EL CONTROLADOR

```
@Post()
create(@Body() dto: CreateUserDto) {
  return this.usersService.create(dto);
}
```

Tipado **@Body() dto: CreateUserDto**: en runtime el pipe garantiza forma y tipos declarados antes de ejecutar la firma.

Anidamiento y colecciones

Para líneas de pedido o colecciones, combiná `@ValidateNested`, `@Type` (de `class-transformer`) y validadores sobre el array (`@IsArray`, `ArrayMinSize`, ...).

```
import { Type } from 'class-transformer';
import {
  ArrayMinSize,
  IsArray,
  IsInt,
  IsPositive,
  IsUUID,
  ValidateNested,
} from 'class-validator';

export class CreateOrderLineInputDto {
  @IsUUID()
  productId: string;

  @IsInt()
  @IsPositive()
  quantity: number;
}

export class CreateOrderDto {
  @IsUUID()
  userId: string;

  @IsArray()
  @ArrayMinSize(1)
  @ValidateNested({ each: true })
  @Type(() => CreateOrderLineInputDto)
  lines: CreateOrderLineInputDto[];
}
```

Decoradores que más vas a usar

No es lista exhaustiva: la librería trae más validadores ("contains", "creditcard", etc.). Esta tabla cubre los casos frecuentes en APIs REST.

TIPO Y FORMA

- `@IsString()` · cadena (`@IsNotEmpty()` si rechazás "").
- `@IsNumber()`, `@IsInt()`, `@IsBoolean()`.
- `@IsEnum(MyEnum)` · literal de enum TypeScript ya compilado.
- `@IsArray()` · array; combinar con `each: true` en otros decoradores donde corresponda.
- `@IsObject()` · objeto plano antes de entrar en `ValidateNested`.

FORMATO Y UNICIDAD SINTÁCTICA

- `@IsEmail()` · formato email (opciones en el decorador).
- `@IsUUID()`, `@IsUrl()`, `@Matches(/regex/)`.
- `@IsISO8601()` · fechas ISO; `@IsDate()` después de parser correcto/coerción.

TAMAÑOS Y LÍMITES

- `@Length(min, max)` texto; `@MinLength(n)`, `@MaxLength(n)`.
- `@Min(n)`, `@Max(n)` números; `@IsPositive()`, `@MinDate()/@MaxDate()`.
- `@ArrayMinSize(n)`, `@ArrayMaxSize(n)`.

PRESENCIA / CONDICIÓN

- `@IsOptional()` · no valida si el valor es `null` / `undefined` (`@IsDefined()` en el caso contrario).
- `@ValidateIf(o ⇒ condición)` · solo aplica los demás decoradores si se cumple (útil cuando un campo depende de otro).

- `@ValidateNested()` · profundiza en objeto o en cada elemento con `@Type`.

BUENAS PRÁCTICAS

Mensajes, i18n y límites

- Podés pasar mensaje por decorador: `@MinLength(8, { message: 'Contraseña demasiado corta' })`.
- Errores estructurados: el pipe devuelve un arreglo; un **ExceptionHandler** puede aplanarlo a un formato { field, constraints } estándar en tu equipo.
- Validás en entrada; igual podés tener **unique**, **checks** y **FK** en BD (los duplicados o violaciones llegan después como error de persistencia).
- Para payloads muy grandes valorá también límites de body en HTTP y rate limiting a nivel servidor.